A Practical Tutorial for Xschem Schematic Capture

Using the GlobalFoundries 180 nm PDK (`gf180mcuD`)

James E. Stine and Landon Burleson
Oklahoma State University
Electrical and Computer Engineering Department
Stillwater, OK 74078 USA
james.stine@okstate.edu

May 7, 2026

Contents

1 Introduction

Xschem is an open-source schematic capture and netlist generation tool originally developed by Stefan Schippers and now maintained at https://xschem.sourceforge.io. It is the de facto schematic editor for the open-source analog/mixed-signal IC design flow, providing fast hierarchical schematic entry, parameterized symbol creation, and direct integration with ngspice, Xyce, and other SPICE-class simulators. Xschem receives frequent updates; always prefer version 3.4 or later.

This tutorial targets the **GlobalFoundries 180 nm MCU PDK** (`gf180mcuD`), an open-source process development kit available through the `open_pdks` installer and supported by Google's open-silicon initiative. The `gf180mcuD` library provides Xschem-ready symbols for nMOS, pMOS, resistors, capacitors, BJTs, and diodes, along with model files for ngspice simulation. All symbol names, parameter conventions, and command examples in this document refer to `gf180mcuD` unless otherwise noted.

Tip

Always sketch your schematic on paper or a whiteboard before opening Xschem. A clean schematic is a planning artifact, not a drawing exercise: spending five minutes thinking through power and ground placement, signal flow, and naming conventions will save you hours of rewiring, deleting, and renaming inside the editor. A good rule is *signals flow left-to-right, supplies run top (VDD) to bottom (GND), and inputs land on the left edge while outputs exit on the right.* Once that mental layout is fixed, schematic entry becomes mechanical.

2 Prerequisites and Setup

2.1 Software

You need Xschem (3.4+), ngspice (40+), and `open_pdks` with `gf180mcuD` installed. If you are using the IIC-OSIC-TOOLS Docker container (https://github.com/iic-jku/iic-osic-tools), all of this is pre-installed and pre-configured—skip ahead to Section 3.

For a native Linux build, follow the accompanying installation guide and confirm that the following environment variables are set (`setup.sh` handles this automatically):

```
export OPEN_CIRCUITDESIGN_ROOT="$HOME/ocd"
export PDK_ROOT="$OPEN_CIRCUITDESIGN_ROOT/share/pdk"
export PDK=gf180mcuD
```

2.2 Environment Check

Before starting any design work, verify that Xschem can see the PDK symbols:

```
ls $PDK_ROOT/gf180mcuD/libs.tech/xschem/
# Should list: gf180mcu, sources, simulator_commands, etc.
```

If the directory is empty or missing, re-run `open_pdks` for `gf180mcuD` or verify that `PDK_ROOT` is set correctly.

2.3 Configuring `xschemrc`

Xschem reads its configuration from an `xschemrc` file, which tells the tool where to find PDK symbol libraries, simulation models, and your personal designs. The PDK ships with one ready to use:

```
ls $PDK_ROOT/gf180mcuD/libs.tech/xschem/xschemrc
```

You can either invoke Xschem with that file directly or copy it to `~/.xschem/xschemrc` and customize the `XSCHEM_LIBRARY_PATH` variable to include your own design directories.

2.4 Create a Working Directory

Organize each design in its own directory to avoid file collisions:

```
mkdir -p ~/designs/inverter
cd ~/designs/inverter
```

3 Launching Xschem

Always launch Xschem with the `gf180mcuD` configuration so that the correct symbol libraries and simulation models are loaded:

```
xschem --rcfile $PDK_ROOT/gf180mcuD/libs.tech/xschem/xschemrc
```

Xschem opens two windows simultaneously:

- **Schematic (drawing) window** — where you place components and draw wires.
- **Terminal (console) window** — the shell where Xschem was launched, which displays netlist generation messages, simulator output, and all error/warning text.

Important

Keep *both* windows visible at all times. Symbol-load failures, parameter parsing errors, and simulator messages appear *only* in the terminal window. The schematic window will look completely normal even when serious problems exist—a missing model file, a malformed parameter, or a simulation that exited immediately leaves no visible artifact in the schematic itself. The *only* indication that something has gone wrong is a message in the terminal. A few practical habits follow from this:

- **After every netlist generation** (`Netlist` menu or pressing `N`), glance at the terminal before continuing.
- **After launching a simulation**, scroll through the full ngspice output rather than stopping at the first "OK" line.
- **When behavior seems wrong**—a component appears blank, a simulation produces no waveforms, or a launch button does nothing—the explanation is almost always already waiting in the terminal window.

Treat the terminal not as a log to be consulted occasionally, but as a live diagnostic stream that runs in parallel with everything you do.

4 The Xschem Interface

4.1 Mouse Buttons

The mouse is your primary schematic instrument. Learn these three actions before anything else:

Button	Shorthand	Action
Left click	L	Place an instance, end a wire, or select an item
Middle click	M	Pan the view; drag with middle-button to pan freely
Right click	R	Open the context menu (properties, delete, copy, etc.)

The mouse wheel zooms in and out about the cursor location. A drag with left-click on empty canvas creates a rubber-band selection rectangle—everything fully enclosed becomes selected.

Tip

Get used to right-clicking on instances and wires. The context menu is the fastest path to editing properties, copying selections, and deleting items, and it always shows you only the operations that are valid for the object under the cursor.

4.2 Entering Commands

Xschem combines several input modes:

1. **Single-character keyboard macros** typed in the schematic window (e.g., `w` starts a wire, `Insert` opens the symbol browser, `f` fits the view).
2. **Menu commands** from the top menu bar (File, Edit, View, Symbol, Simulation, etc.).
3. **Mouse actions** for placement, selection, and panning.

The most important macros to learn first are `Insert` (place a new instance), `w` (wire), `l` (label/wire name), `Escape` (cancel current operation), and `Ctrl+s` (save).

4.3 Selection and Movement

Click to select a single object. Hold `Shift` and click to add to the selection, or drag a rubber-band rectangle to select a region. Move selected items with `m`, copy them with `c`, and delete them with `Delete`. Press `Escape` at any time to cancel the current operation without committing changes.

Important

A common source of confusion is starting a wire with `w` and then accidentally beginning to type a label or instance name into the schematic window. Xschem captures every keystroke as a potential macro, so what you *think* is text entry can instead trigger another command—moving, deleting, or rotating the selection unexpectedly. If something stops working as expected—a click produces no result, or the cursor seems stuck in some other mode—press `Escape` once or twice to return to the default state before doing anything else. Make it a habit to press `Escape` between distinct operations, particularly after wire drawing, label placement, or property editing.

5 Essential Command Reference

5.1 File and View Commands

Command/Key	Description
`Ctrl+s`	Save current schematic
`Ctrl+o`	Open existing schematic
`Ctrl+n`	New empty schematic in a fresh window
`Ctrl+q`	Quit Xschem
`f`	Fit all geometry to window (zoom-fit)
`z` / `Z`	Zoom in / zoom out about the cursor
`Ctrl+e`	Edit (descend into) the selected instance's schematic
`Ctrl+i`	Edit (descend into) the selected instance's symbol
`Ctrl+a`	Select all
`Esc`	Cancel current operation

5.2 Drawing and Editing

Action	Macro	Description
Place new instance	`Insert`	Open symbol browser to place a component
Draw wire	`w`	Start a wire; left-click to add vertices
Place label/wire name	`l`	Insert a net label (`lab_pin`)
Place text annotation	`t`	Add free-form text (non-electrical)
Move selection	`m`	Move selected items
Copy selection	`c`	Copy selected items
Rotate selection	`r`	Rotate 90° counter-clockwise
Flip selection	`Shift+f`	Mirror selection
Delete selection	`Delete`	Remove selected items
Edit properties	`q`	Open property dialog for selected instance

5.3 Netlist and Simulation

Macro/Menu	Description
`Netlist`	Generate a SPICE netlist for the current schematic
`Simulate`	Launch the configured simulator (ngspice by default) on the netlist
`a`	Show/hide net annotations after a simulation
`Edit netlist`	Open the most recent netlist in your editor for inspection

The `Options` menu lets you switch netlist format between SPICE, Verilog, VHDL, and tEDAx. For `gf180mcuD` simulation work, leave it on SPICE.

6 gf180mcuD Symbol Reference

The PDK provides Xschem symbols for every primitive device. The most commonly used symbols for digital and mixed-signal cell design are listed below; the *Symbol path* is what appears in the `Insert` symbol browser.

Symbol Path	Description
`gf180mcu/nfet_03v3`	Standard 3.3 V nMOS transistor
`gf180mcu/pfet_03v3`	Standard 3.3 V pMOS transistor
`gf180mcu/nfet_06v0`	5.0 V (6 V tolerant) nMOS
`gf180mcu/pfet_06v0`	5.0 V (6 V tolerant) pMOS
`gf180mcu/res_*`	Resistor primitives (poly, metal, diffusion)
`gf180mcu/cap_*`	Capacitor primitives (MIM, MOS, MOM)
`gf180mcu/diode_*`	Diode primitives
`devices/ipin`	Schematic input port (left edge of cell)
`devices/opin`	Schematic output port (right edge of cell)
`devices/iopin`	Bidirectional port
`devices/lab_pin`	Net-name label
`devices/gnd`	Ground symbol
`devices/vsource`	Voltage source
`devices/code_shown`	Inline SPICE code block (for `.tran`, `.include`, etc.)

Tip

Use the right transistor flavor for your supply. `gf180mcuD` provides separate device families for 3.3 V and 5 V operation. Mixing them in a single design is allowed—and often necessary for mixed-signal interfaces—but each device must operate within its rated V_{ds}, V_{gs}, and V_{bs} limits or it will degrade quickly in silicon. For most digital cell work in `gf180mcuD`, the 3.3 V family (`nfet_03v3` and `pfet_03v3`) is the correct choice. Reach for the 5 V devices only when an interface specifically requires them.

7 Ports, Labels, and Hierarchy

7.1 Ports vs. Labels

Xschem distinguishes between two kinds of named connections:

- **Ports** (`ipin`, `opin`, `iopin`) define the external interface of a cell. When you generate a symbol from this schematic, every port becomes a pin on the symbol.
- **Labels** (`lab_pin`) name internal nets so they can be referred to by name elsewhere in the same schematic, or so they appear meaningfully in the netlist.

A single net can carry multiple labels with the same name; Xschem treats them as electrically connected even without a visible wire. This is the standard way to wire up power and ground rails without cluttering the schematic.

7.2 Naming Conventions

- `VDD` and `VSS` (or `GND`) — supply rails. Use the same names as in your testbench and any other cells you intend to instantiate.
- Signal names must match between the schematic and its companion layout for LVS to pass; use exactly the same capitalization.
- Adopt a consistent style within a project—`a`, `b`, `y`, `clk`, `rst`, etc.—and keep it everywhere.

Tip

Match labels to the schematic of the surrounding system. A net labeled `Y` in your inverter must connect to a net also labeled `Y` in any cell that uses it. LVS, top-level netlist generation, and inter-cell wiring all depend on character-for-character label matches. A net labeled `vdd` will not connect to a net labeled `VDD`, and Xschem will not warn you about the mismatch—it will simply produce a netlist with two disconnected supplies.

7.3 Creating a Symbol from a Schematic

Every schematic with ports can be turned into a symbol that other schematics can instantiate. With your schematic open, choose **Symbol → Make symbol from schematic** (or press `a` from the menu shortcut). Xschem auto-generates a rectangular symbol with one pin per port, saved as `<name>.sym` alongside your `<name>.sch`. After that, instantiate it from another schematic with `Insert` and browse to your working directory.

7.4 Descending and Ascending the Hierarchy

Select an instance and press `Ctrl+e` to descend into its schematic, or `Ctrl+i` to descend into its symbol drawing. Press `Ctrl+e` again on empty canvas (or use the `<<` button on the toolbar) to ascend back to the parent.

8 Step-by-Step: CMOS Inverter Schematic in gf180mcuD

This section walks through drawing a minimum-size CMOS inverter from scratch in `gf180mcuD`. Have your sketch ready before you begin.

8.1 Transistor Sizing

In `gf180mcuD` the minimum drawn gate length is $L_{\min} = 0.18\,\mu$m. A reasonable starting inverter uses:

Transistor	Type	Width	Length
nMOS	`nfet_03v3`	$0.42\,\mu$m	$0.18\,\mu$m
pMOS	`pfet_03v3`	$0.84\,\mu$m ($2\times$ nMOS)	$0.18\,\mu$m

The pMOS is sized $2\times$ wider than the nMOS to equalize rise and fall times, since hole mobility is roughly half that of electrons.

8.2 Step 1 — Launch Xschem and Create a New Schematic

```
cd ~/designs/inverter
xschem --rcfile $PDK_ROOT/gf180mcuD/libs.tech/xschem/xschemrc
```

In the schematic window, press `Ctrl+n` to open a new empty schematic. Save it immediately so all subsequent operations have a target file:

```
File -> Save As ...   # save as inv.sch in ~/designs/inverter
```

8.3 Step 2 — Place the nMOS Transistor

Press `Insert` to open the symbol browser, then navigate to `gf180mcu/nfet_03v3` and double-click to attach it to the cursor. Left-click in the lower half of the canvas to place it. Press `Escape` to exit placement mode.

Select the placed instance, press `q` to open its properties, and set:

```
W = 0.42u
L = 0.18u
nf = 1
```

8.4 Step 3 — Place the pMOS Transistor

Press `Insert` again, navigate to `gf180mcu/pfet_03v3`, and place it directly above the nMOS. Edit its properties (`q`):

```
W = 0.84u
L = 0.18u
nf = 1
```

8.5 Step 4 — Place Power, Ground, and Substrate Connections

A MOSFET symbol in the PDK has four terminals: drain, gate, source, and body. The body terminals must be tied to the appropriate supply for correct operation:

- nMOS body → `VSS` (ground).
- pMOS body (n-well) → `VDD`.

The simplest way to make these connections is to label the body wires with `VDD` and `VSS` so they connect by name instead of running explicit wires.

8.6 Step 5 — Draw the Wires

Press `w` to enter wire mode and connect:

- pMOS source → `VDD` rail.
- nMOS source → `VSS` rail.
- pMOS drain + nMOS drain → output node.
- pMOS gate + nMOS gate → input node.

Click each vertex of the wire path; double-click or press `Esc` to finish. Wires snap to the grid automatically and form T-junctions wherever they cross other wires at vertices.

Tip

Always wire on grid. Xschem will not connect two wires that pass each other off-grid, even if they look connected on screen. If a node refuses to take a label or shows up disconnected after netlisting, the most common cause is one endpoint sitting half a grid unit off. Press `f` to fit, then zoom in (`z`) on the suspected junction and verify both endpoints are exactly on grid intersections.

8.7 Step 6 — Add Port Symbols

Press `Insert`, browse to `devices/ipin`, and place an input pin on the input wire. Edit its properties (`q`) and set:

```
name = A
```

Repeat with `devices/opin` on the output wire, naming it `Y`. These ports define the external interface of the cell—when you generate a symbol later, `A` and `Y` become the input and output pins.

8.8 Step 7 — Add Labels for Power and Ground

Press `Insert`, browse to `devices/lab_pin`, and place labels on the supply wires:

```
name = VDD       # on the upper rail and pMOS body
name = VSS       # on the lower rail and nMOS body
```

Anywhere two `lab_pin` instances share the same name, Xschem treats the wires they sit on as electrically connected—even with no visible wire between them.

8.9 Step 8 — Save

```
Ctrl+s
```

This writes `inv.sch` to your working directory. Save frequently; Xschem does not auto-save.

8.10 Step 9 — Generate the Symbol

With the schematic saved, choose **Symbol → Make symbol from schematic**. Xschem creates `inv.sym` in the same directory, ready to be instantiated by a testbench or higher-level cell.

Important

If you later add, remove, or rename ports, regenerate the symbol. A schematic and its symbol can drift out of sync silently—an `ipin` added to the schematic will not appear on the symbol until you regenerate it, and any cell that already instantiated the old symbol will continue to use the old pin list with no warning. Make symbol regeneration part of your save workflow whenever the port list changes.

9 Building a Testbench

A testbench schematic instantiates the cell under test, applies stimulus, and includes the simulator directives needed to run a particular analysis. This section builds a minimal inverter testbench.

9.1 Step 1 — New Schematic

Start a fresh schematic with `Ctrl+n` and save it as `inv_tb.sch` in the same directory as `inv.sch`.

9.2 Step 2 — Instantiate the Inverter

Press `Insert` and browse to your working directory rather than the PDK library. Select `inv.sym` and place it in the center of the canvas.

9.3 Step 3 — Add the Supply

Place a `devices/vsource` symbol, set its properties to `value = 3.3` and `name = VDD`, and wire its positive terminal to a label `VDD` and its negative terminal to a label `VSS`. Place a `devices/gnd` symbol on the `VSS` node so that ngspice has a defined zero-volt reference.

9.4 Step 4 — Add the Input Stimulus

Place another `devices/vsource` on input `A`. Edit its properties:

```
name  = VIN
value = pulse(0 3.3 0 100p 100p 5n 10n)
```

This drives `A` with a 100 MHz pulse train (10 ns period, 50% duty cycle, 100 ps edges). The negative terminal connects to `VSS`.

9.5 Step 5 — Add Simulator Directives

Place a `devices/code_shown` symbol anywhere on the canvas. Edit its `value` property to contain the simulation control deck:

```
.lib "$PDK_ROOT/gf180mcuD/libs.tech/ngspice/design.ngspice" typical
.tran 10p 50n
.save all
.control
run
plot v(A) v(Y)
.endc
```

The first line includes the typical-corner device models for `gf180mcuD`. The `.tran` directive runs a 50 ns transient with 10 ps internal time steps, the `.save all` stores every node, and the `.control` block automatically plots the input and output once the simulation finishes.

9.6 Step 6 — Generate the Netlist

Press `N` (or use **Simulation** → **Netlist**). Xschem writes `inv_tb.spice` to the working directory and prints any errors to the terminal.

9.7 Step 7 — Run the Simulation

Press the **Simulate** button (or use the **Simulation** menu). Xschem launches ngspice on the netlist; ngspice runs in a console and opens a plot window with the requested waveforms.

Tip

Re-netlist before every simulation. Xschem does not automatically regenerate the netlist when you change the schematic. A common debugging trap is to edit the schematic, hit Simulate, and see exactly the same (broken) result as before—because ngspice ran the old netlist. Make `N` (netlist) a reflex before `Simulate`, or use the **Netlist + Simulate** menu option that does both in one step.

10 Netlist Generation in Detail

The netlist Xschem generates is a plain-text SPICE file usable directly by ngspice, Xyce, or any other Berkeley-compatible simulator. You can inspect it with any text editor:

```
cat inv_tb.spice
```

Useful options under the **Options** menu:

- **Spice netlist**: standard SPICE output (default).
- **Hierarchical netlist**: emits subcircuit definitions (`.subckt`) for every cell rather than flattening. Required for any design with more than one level of hierarchy.
- **Top-level is subckt**: wraps the top schematic itself in a `.subckt` block so it can be reused. Leave this off for testbenches.

Important

For LVS-clean cell design, always use the hierarchical netlister. A flattened netlist destroys the cell-by-cell structure that LVS tools rely on to compare your schematic against the layout. Magic's `.spice` extraction is hierarchical by default; your schematic netlist must match.

11 Simulation with ngspice

When Xschem launches ngspice, you get an interactive console where you can issue additional commands after the initial `.control` block finishes:

```
ngspice 1 -> plot v(A) v(Y)
ngspice 2 -> meas tran t_pdr trig v(A) val=1.65 fall=1 \
                              targ v(Y) val=1.65 rise=1
ngspice 3 -> meas tran t_pdf trig v(A) val=1.65 rise=1 \
                              targ v(Y) val=1.65 fall=1
ngspice 4 -> let t_pd = (t_pdr + t_pdf) / 2
ngspice 5 -> print t_pd
```

These four `meas` statements compute the rising-edge propagation delay (`t_pdr`), falling-edge propagation delay (`t_pdf`), and their average (`t_pd`) for the inverter.

11.1 Common Analyses

Directive	Purpose
`.tran 10p 50n`	Transient: 10 ps step, 50 ns stop
`.dc VIN 0 3.3 0.01`	DC sweep of `VIN` from 0 to 3.3 V in 10 mV steps
`.ac dec 10 1 1G`	AC sweep, 10 points per decade, 1 Hz to 1 GHz
`.noise v(out) VIN dec 10 1 1G`	Noise analysis at output referred to `VIN`
`.op`	DC operating point only

For a simple voltage-transfer-curve plot of an inverter, use `.dc` sweeping the input source rather than `.tran`. Pulse stimuli can be useful but are unnecessary for static characterization.

12 Schematic vs. Layout (LVS Workflow)

Once your schematic simulates correctly, the SPICE netlist Xschem generates becomes the reference for Layout vs. Schematic (LVS) comparison against the corresponding Magic layout. Many professional organizations list skipping LVS as a termination offense.

A typical workflow:

1. In Xschem, generate a hierarchical netlist of the cell (*not* the testbench): `Options` → `Hierarchical netlist`, then `Netlist`.
2. In Magic, extract the layout to SPICE (`:extract all; :ext2spice`).
3. Run Netgen with the PDK setup file:

```
netgen -batch lvs "inv.spice inv" \
    "$PDK_ROOT/gf180mcuD/libs.tech/netgen/gf180mcuD_setup.tcl"
```

4. A passing comparison prints **Circuits match**. A failure lists mismatched devices and nets—trace each one back to a label or wiring error in either side.

Tip

The schematic is the *golden* reference. If LVS fails, the layout is what changes—never edit the schematic to "match" a faulty layout, since the schematic is what your simulation verified. The only exception is when the schematic itself is wrong relative to the design intent, in which case you fix the schematic, re-simulate, re-netlist, and re-run LVS from the top.

13 Tips and Common Pitfalls

- **Re-netlist before every simulation.** Xschem does not automatically regenerate the netlist when you edit the schematic. Press `N` before `Simulate` every time, or use the combined menu option.
- **Match labels exactly.** `VDD` and `vdd` are different nets. Pick a convention (typically all-uppercase for supplies and lowercase for signals) and follow it everywhere.
- **Tie every body terminal.** A floating MOSFET body produces nonsense simulation results and an LVS failure. Always label the body net to `VDD` (pMOS) or `VSS` (nMOS) explicitly.
- **Keep wires on grid.** Off-grid wires look connected on screen but produce disconnected nets in the netlist. When in doubt, zoom in and verify each junction.
- **Regenerate the symbol after port changes.** Adding, removing, or renaming `ipin`/`opin` instances does not automatically update the symbol file. Use **Symbol** → **Make symbol from schematic** whenever the port list changes.
- **Use `code_shown` for SPICE directives.** Inline SPICE blocks (`.tran`, `.lib`, `.control`) belong in a `code_shown` symbol on the testbench schematic, not buried in the netlist.
- **Save your testbench separately.** Keep `<cell>.sch` and `<cell>_tb.sch` as independent files in the same directory. The cell schematic stays clean and reusable; the testbench carries the simulation setup.
- **Read the terminal.** Almost every diagnostic Xschem and ngspice produce appears in the launching terminal, not in the GUI. Make a habit of glancing at it after every netlist and simulate.
- **Use scripts.** Repeated simulation runs, sweep generation, and corner-by-corner netlisting are all scriptable from the Tcl console (open with **Window** → **Tcl prompt**). A small library of reusable Tcl snippets pays for itself quickly.

14 Quick-Reference Cheat Sheet

Keyboard Macros

Key	Action	Notes
`Insert`	Place new instance	Opens symbol browser
`w`	Draw wire	Click vertices; `Esc` to end
`l`	Place label	Insert `lab_pin`
`t`	Place text	Non-electrical annotation
`m`	Move selection	—
`c`	Copy selection	—
`r`	Rotate 90° CCW	—
`Shift+f`	Flip	Mirror selection
`q`	Edit properties	Of selected instance
`Delete`	Delete selection	—
`f`	Fit to window	—
`z/Z`	Zoom in/out	About cursor
`Ctrl+e`	Descend into schematic	Of selected instance
`Ctrl+i`	Descend into symbol	Of selected instance
`Ctrl+s`	Save	—
`Ctrl+a`	Select all	—
`Esc`	Cancel	Always safe

Key Commands

To launch Xschem with the gf180mcuD configuration:

```
xschem --rcfile $PDK_ROOT/gf180mcuD/libs.tech/xschem/xschemrc
```

Alternatively, copy the PDK `xschemrc` to `~/.xschem/xschemrc` so Xschem loads it automatically every time:

```
mkdir -p ~/.xschem
cp $PDK_ROOT/gf180mcuD/libs.tech/xschem/xschemrc ~/.xschem/xschemrc
```

With this in place, simply typing `xschem` from any directory will load the `gf180mcuD` symbol libraries without requiring the `--rcfile` argument each time.

Menu/Action	Purpose
`Netlist`	Generate SPICE netlist for current schematic
`Simulate`	Run ngspice on the netlist
`Symbol` → `Make symbol`	Generate `.sym` from `.sch`
`Options` → `Hierarchical`	Toggle hierarchical netlister (required for LVS)
`Window` → `Tcl prompt`	Open the Tcl scripting console

Common Symbols for gf180mcuD

Symbol path	What it places
`gf180mcu/nfet_03v3`	3.3 V nMOS
`gf180mcu/pfet_03v3`	3.3 V pMOS
`gf180mcu/res_*`	Resistor primitives
`gf180mcu/cap_*`	Capacitor primitives
`devices/ipin`	Input port
`devices/opin`	Output port
`devices/iopin`	Bidirectional port
`devices/lab_pin`	Net label
`devices/gnd`	Ground
`devices/vsource`	Voltage source
`devices/code_shown`	SPICE directive block

Additional Resources

- Xschem manual and tutorials: https://xschem.sourceforge.io/stefan/index.html
- ngspice manual: https://ngspice.sourceforge.io/docs.html
- gf180mcuD PDK documentation: https://gf180mcu-pdk.readthedocs.io
- open_pdks repository (PDK installer): https://github.com/RTimothyEdwards/open_pdks
- IIC-OSIC-TOOLS Docker container: https://github.com/iic-jku/iic-osic-tools
- Course examples, scripts, and schematic templates: https://stineje.github.io
- Video walkthroughs: https://www.youtube.com/user/jlstine/